
SERVICIO NACIONAL DE APRENDIZAJE - SENA

Regional Boyacá

Programa: Análisis y Desarrollo de Software (ADSO)

TALLER PRÁCTICO

Programación Orientada a Objetos en Java

Los Cuatro Pilares Fundamentales

Entorno:	IDE NetBeans
Lenguaje:	Java (JDK 17+)
Duración:	6 horas
Modalidad:	Presencial - Práctico

1. Introducción

La Programación Orientada a Objetos (POO) es un paradigma fundamental en el desarrollo de software moderno. Java, como lenguaje orientado a objetos por excelencia, implementa los cuatro pilares de la POO de manera robusta y clara. En este taller, exploraremos cada pilar con explicaciones teóricas y ejercicios prácticos desarrollados en NetBeans.

1.1 Objetivo General

Comprender y aplicar los cuatro pilares fundamentales de la Programación Orientada a Objetos (Abstracción, Encapsulamiento, Herencia y Polimorfismo) mediante ejercicios prácticos en Java utilizando el IDE NetBeans.

1.2 Objetivos Específicos

- Modelar entidades del mundo real aplicando el principio de abstracción.
- Proteger los datos de las clases mediante encapsulamiento con modificadores de acceso y métodos getter/setter.
- Reutilizar código mediante herencia, creando jerarquías de clases coherentes.
- Implementar polimorfismo para lograr comportamientos dinámicos en tiempo de ejecución.

1.3 Requisitos Previos

- JDK 17 o superior instalado y configurado.
- IDE Apache NetBeans 17+ instalado.
- Conocimientos básicos de sintaxis Java (variables, condicionales, ciclos, métodos).

Configuración en NetBeans

Para cada ejercicio, cree un nuevo proyecto Java: File > New Project > Java Application. Nombre el proyecto según el pilar correspondiente (ej: TallerAbstraccion). Cada clase se crea con: Click derecho sobre el paquete > New > Java Class.

2. Pilar 1: Abstracción

Definición

La abstracción consiste en identificar las características esenciales de un objeto, ignorando los detalles irrelevantes. Permite modelar entidades del mundo real centrándose en QUÉ hace un objeto, no en CÓMO lo hace.

En Java, la abstracción se implementa mediante clases abstractas e interfaces. Una clase abstracta define un contrato parcial que las subclasses deben completar, mientras que una interfaz define un contrato completo de comportamientos.

2.1 Clases Abstractas

Una clase abstracta no puede ser instanciada directamente. Puede contener métodos abstractos (sin implementación) y métodos concretos (con implementación).

Ejemplo: Sistema de Figuras Geométricas

Cree un proyecto llamado TallerAbstraccion y agregue las siguientes clases:

Clase: FiguraGeometrica.java

```
public abstract class FiguraGeometrica {
    protected String nombre;
    protected String color;

    public FiguraGeometrica(String nombre, String color) {
        this.nombre = nombre;
        this.color = color;
    }

    // Métodos abstractos - las subclasses DEBEN implementarlos
    public abstract double calcularArea();
    public abstract double calcularPerimetro();

    // Método concreto - compartido por todas las subclasses
    public void mostrarInfo() {
        System.out.println("Figura: " + nombre);
        System.out.println("Color: " + color);
        System.out.printf("Area: %.2f%n", calcularArea());
        System.out.printf("Perimetro: %.2f%n", calcularPerimetro());
    }
}
```

Clase: Circulo.java

```
public class Circulo extends FiguraGeometrica {
    private double radio;

    public Circulo(String color, double radio) {
```

```
        super("Círculo", color);
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * Math.PI * radio;
    }
}
```

Clase: Rectangulo.java

```
public class Rectangulo extends FiguraGeometrica {
    private double base;
    private double altura;

    public Rectangulo(String color, double base, double altura) {
        super("Rectángulo", color);
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return base * altura;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * (base + altura);
    }
}
```

Clase: Main.java

```
public class Main {
    public static void main(String[] args) {
        FiguraGeometrica circulo = new Circulo("Rojo", 5.0);
        FiguraGeometrica rectangulo = new Rectangulo("Azul", 4.0, 6.0);

        circulo.mostrarInfo();
        System.out.println("---");
        rectangulo.mostrarInfo();
    }
}
```

2.2 Interfaces

Las interfaces definen contratos de comportamiento que las clases pueden implementar. Una clase puede implementar múltiples interfaces, lo que ofrece mayor flexibilidad que la herencia simple.

Interface: Imprimible.java

```
public interface Imprimible {  
    void imprimir();  
    String formatoResumen();  
}
```

Interface: Exportable.java

```
public interface Exportable {  
    String exportarCSV();  
}
```

Clase: Producto.java (implementa múltiples interfaces)

```
public class Producto implements Imprimible, Exportable {  
    private String nombre;  
    private double precio;  
  
    public Producto(String nombre, double precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
  
    @Override  
    public void imprimir() {  
        System.out.println(formatoResumen());  
    }  
  
    @Override  
    public String formatoResumen() {  
        return String.format("Producto: %s - $%,.0f COP", nombre, precio);  
    }  
  
    @Override  
    public String exportarCSV() {  
        return nombre + "," + precio;  
    }  
}
```

Ejercicio Práctico 1 - Abstracción

Cree una clase abstracta Vehiculo con atributos marca y modelo, y métodos abstractos calcularConsumo(double km) y mostrarFichaTecnica(). Implemente las subclases Automovil (con atributo cilindraje) y Motocicleta (con atributo tipo: deportiva/urbana). Cada subclase debe implementar los métodos abstractos con lógica propia. Cree una clase Main que instancie al menos 2 vehículos y muestre sus fichas técnicas.

3. Pilar 2: Encapsulamiento

Definición

El encapsulamiento es el mecanismo que protege los datos internos de un objeto, restringiendo el acceso directo y proporcionando métodos controlados (getters y setters) para interactuar con ellos. Garantiza la integridad de los datos.

3.1 Modificadores de Acceso en Java

Modificador	Clase	Paquete	Subclase	Mundo
public	Sí	Sí	Sí	Sí
protected	Sí	Sí	Sí	No
default (sin mod.)	Sí	Sí	No	No
private	Sí	No	No	No

3.2 Ejemplo Práctico: Cuenta Bancaria

Este ejemplo demuestra cómo proteger datos sensibles como el saldo de una cuenta bancaria, validando las operaciones antes de modificar el estado interno.

Clase: CuentaBancaria.java

```
public class CuentaBancaria {
    private String titular;
    private String numeroCuenta;
    private double saldo;
    private String clave;

    public CuentaBancaria(String titular, String numeroCuenta,
                           double saldoInicial, String clave) {
        this.titular = titular;
        this.numeroCuenta = numeroCuenta;
        this.saldo = saldoInicial;
        this.clave = clave;
    }

    public String getTitular() {
        return titular;
    }

    // Getter controlado - muestra solo los ultimos 4 digitos
    public String getNumeroCuentaOculto() {
        return "****" + numeroCuenta.substring(
            numeroCuenta.length() - 4);
    }
}
```

```

    }

    // Getter del saldo - requiere autenticacion
    public double consultarSaldo(String claveIngresada) {
        if (autenticar(claveIngresada)) {
            return saldo;
        }
        System.out.println("Clave incorrecta.");
        return -1;
    }

    public boolean depositar(double monto) {
        if (monto <= 0) {
            System.out.println("El monto debe ser positivo.");
            return false;
        }
        saldo += monto;
        System.out.printf("Deposito exitoso: $%,.0f COP%n", monto);
        return true;
    }

    public boolean retirar(double monto, String claveIngresada) {
        if (!autenticar(claveIngresada)) {
            System.out.println("Clave incorrecta.");
            return false;
        }
        if (monto <= 0 || monto > saldo) {
            System.out.println("Monto invalido o saldo insuficiente.");
            return false;
        }
        saldo -= monto;
        System.out.printf("Retiro exitoso: $%,.0f COP%n", monto);
        return true;
    }

    // Metodo privado - solo accesible dentro de la clase
    private boolean autenticar(String claveIngresada) {
        return this.clave.equals(claveIngresada);
    }

    @Override
    public String toString() {
        return String.format("Titular: %s | Cuenta: %s",
            titular, getNumeroCuentaOculto());
    }
}

```

Clase: Main.java

```

public class Main {
    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria(
            "Carlos Perez", "1234567890", 500000, "miClave123");

        System.out.println(cuenta);
        cuenta.depositar(200000);
    }
}

```

```
cuenta.retirar(100000, "miClave123");
cuenta.retirar(100000, "claveErronea"); // Falla

double saldo = cuenta.consultarSaldo("miClave123");
System.out.printf("Saldo actual: $%,.0f COP%n", saldo);
}
}
```

Ejercicio Práctico 2 - Encapsulamiento

Cree una clase Estudiante con atributos privados: nombre, codigo, notas (array de 3 doubles) y estado (aprobado/reprobado). Implemente: getters para todos los atributos, un setter para notas que valide que estén entre 0.0 y 5.0, un metodo calcularPromedio() que retorne el promedio de las notas, y un metodo privado actualizarEstado() que se invoque automaticamente al modificar notas (aprobado si promedio >= 3.0). En el Main, cree 3 estudiantes e intente asignar notas invalidas para verificar la validacion.

4. Pilar 3: Herencia

Definición

La herencia permite crear nuevas clases (hijas) que heredan atributos y métodos de clases existentes (padres). Promueve la reutilización de código y establece relaciones jerárquicas entre las clases.

En Java se utiliza la palabra clave extends para establecer herencia. Java soporta herencia simple (una clase solo puede extender de una clase padre). La clase hija hereda todos los miembros no privados de la clase padre.

4.1 Ejemplo: Sistema de Empleados

Modelaremos un sistema de gestión de empleados con una jerarquía de clases que demuestra la herencia y el uso de super.

Clase Padre: Empleado.java

```
public class Empleado {
    protected String nombre;
    protected String cedula;
    protected double salarioBase;

    public Empleado(String nombre, String cedula, double salarioBase) {
        this.nombre = nombre;
        this.cedula = cedula;
        this.salarioBase = salarioBase;
    }

    public double calcularSalario() {
        return salarioBase;
    }

    public void mostrarInformacion() {
        System.out.println("=== Informacion del Empleado ===");
    }
}
```



```
        System.out.println("Nombre: " + nombre);
        System.out.println("Cedula: " + cedula);
        System.out.printf("Salario: $%,.0f COP%n", calcularSalario());
    }
}
```

Clase Hija: EmpleadoTiempoCompleto.java

```
public class EmpleadoTiempoCompleto extends Empleado {
    private double bonificacion;
    private double auxilioTransporte;

    public EmpleadoTiempoCompleto(String nombre, String cedula,
        double salarioBase, double bonificacion,
        double auxilioTransporte) {
        super(nombre, cedula, salarioBase);
        this.bonificacion = bonificacion;
        this.auxilioTransporte = auxilioTransporte;
    }

    @Override
    public double calcularSalario() {
        return salarioBase + bonificacion + auxilioTransporte;
    }

    @Override
    public void mostrarInformacion() {
        super.mostrarInformacion();
        System.out.printf("Bonificacion: $%,.0f COP%n", bonificacion);
        System.out.printf("Aux. Transporte: $%,.0f COP%n",
            auxilioTransporte);
    }
}
```

Clase Hija: EmpleadoPorHoras.java

```
public class EmpleadoPorHoras extends Empleado {
    private int horasTrabajadas;
    private double valorHora;

    public EmpleadoPorHoras(String nombre, String cedula,
        int horasTrabajadas, double valorHora) {
        super(nombre, cedula, 0);
        this.horasTrabajadas = horasTrabajadas;
        this.valorHora = valorHora;
    }

    @Override
    public double calcularSalario() {
        double salario = horasTrabajadas * valorHora;
        if (horasTrabajadas > 160) {
            int extra = horasTrabajadas - 160;
            salario += extra * valorHora * 0.25;
        }
        return salario;
    }
}
```

```
@Override
public void mostrarInformacion() {
    super.mostrarInformacion();
    System.out.println("Horas trabajadas: " + horasTrabajadas);
    System.out.printf("Valor hora: $%,.0f COP%n", valorHora);
}
}
```

Clase: Main.java

```
public class Main {
    public static void main(String[] args) {
        EmpleadoTiempoCompleto emp1 = new EmpleadoTiempoCompleto(
            "Ana Gomez", "123456789",
            2500000, 300000, 162000);

        EmpleadoPorHoras emp2 = new EmpleadoPorHoras(
            "Luis Martinez", "987654321", 180, 25000);

        emp1.mostrarInformacion();
        System.out.println();
        emp2.mostrarInformacion();
    }
}
```

Ejercicio Práctico 3 - Herencia

Cree una jerarquía para un sistema de gestión de una biblioteca: Clase padre `MaterialBiblioteca` con atributos `titulo`, `autor`, `anioPublicacion` y método `mostrarDetalle()`. Clases hijas: `Libro` (con atributos `numPaginas` e `isbn`), `Revista` (con atributos `edicion` y `periodicidad`), y `MaterialDigital` (con atributos `formato` y `tamanoMB`). Cada clase hija debe sobrescribir `mostrarDetalle()` llamando a `super` y agregando sus datos propios. En el `Main`, cree un `ArrayList<MaterialBiblioteca>` con al menos 5 materiales mixtos y muéstrellos con un ciclo `for-each`.

5. Pilar 4: Polimorfismo

Definición

El polimorfismo permite que objetos de diferentes clases respondan de manera distinta al mismo mensaje (llamada a método). Existen dos tipos: polimorfismo en tiempo de compilación (sobrecarga) y polimorfismo en tiempo de ejecución (sobreescripción).

5.1 Sobrecarga de Métodos (Compile-time)

La sobrecarga ocurre cuando una clase tiene múltiples métodos con el mismo nombre pero diferente firma (parámetros distintos). El compilador decide cuál método invocar según los argumentos.

Clase: Calculadora.java

```
public class Calculadora {

    public int sumar(int a, int b) {
        return a + b;
    }

    public double sumar(double a, double b) {
        return a + b;
    }

    public int sumar(int a, int b, int c) {
        return a + b + c;
    }

    public String sumar(String a, String b) {
        return a + " " + b;
    }

}
```

5.2 Sobreescritura de Métodos (Runtime)

La sobreescritura ocurre cuando una clase hija redefine un método heredado de la clase padre. La JVM decide en tiempo de ejecución cuál implementación ejecutar según el tipo real del objeto.

Ejemplo Integrador: Sistema de Notificaciones

Este ejemplo combina todos los pilares e ilustra el poder del polimorfismo en un escenario real.

Clase Abstracta: Notificacion.java

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public abstract class Notificacion {
    private String destinatario;
    private String mensaje;
    private LocalDateTime fechaCreacion;

    public Notificacion(String destinatario, String mensaje) {
        this.destinatario = destinatario;
        this.mensaje = mensaje;
        this.fechaCreacion = LocalDateTime.now();
    }

    public abstract boolean enviar();
    public abstract double calcularCosto();

    public String getDestinatario() { return destinatario; }
    public String getMensaje() { return mensaje; }

    public String getFechaFormateada() {
        DateTimeFormatter fmt = DateTimeFormatter.ofPattern(
```

```
        "dd/MM/yyyy HH:mm");  
        return fechaCreacion.format(fmt);  
    }  
}
```

Clase: NotificacionEmail.java

```
public class NotificacionEmail extends Notificacion {  
    private String asunto;  
    private boolean tieneAdjunto;  
  
    public NotificacionEmail(String destinatario, String mensaje,  
        String asunto, boolean tieneAdjunto) {  
        super(destinatario, mensaje);  
        this.asunto = asunto;  
        this.tieneAdjunto = tieneAdjunto;  
    }  
  
    @Override  
    public boolean enviar() {  
        System.out.printf("[EMAIL] Enviando a: %s%n",  
            getDestinatario());  
        System.out.println("Asunto: " + asunto);  
        System.out.println("Mensaje: " + getMensaje());  
        if (tieneAdjunto) {  
            System.out.println("(Con archivo adjunto)");  
        }  
        return true;  
    }  
  
    @Override  
    public double calcularCosto() {  
        return tieneAdjunto ? 150.0 : 50.0;  
    }  
}
```

Clase: NotificacionSMS.java

```
public class NotificacionSMS extends Notificacion {  
    private String operador;  
  
    public NotificacionSMS(String destinatario, String mensaje,  
        String operador) {  
        super(destinatario, mensaje);  
        this.operador = operador;  
    }  
  
    @Override  
    public boolean enviar() {  
        System.out.printf("[SMS] Enviando a: %s (%s)%n",  
            getDestinatario(), operador);  
        System.out.println("Mensaje: " + getMensaje());  
        return true;  
    }  
  
    @Override
```

```

    public double calcularCosto() {
        int caracteres = Math.min(getMensaje().length(), 160);
        return caracteres * 5.0;
    }
}

```

Clase: NotificacionPush.java

```

public class NotificacionPush extends Notificacion {
    private String aplicacion;

    public NotificacionPush(String destinatario, String mensaje,
        String aplicacion) {
        super(destinatario, mensaje);
        this.aplicacion = aplicacion;
    }

    @Override
    public boolean enviar() {
        System.out.printf("[PUSH] App '%s' -> %s%n",
            aplicacion, getDestinatario());
        System.out.println("Mensaje: " + getMensaje());
        return true;
    }

    @Override
    public double calcularCosto() {
        return 10.0;
    }
}

```

Clase Main (demuestra polimorfismo): Main.java

```

import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Notificacion> notificaciones = new ArrayList<>();

        notificaciones.add(new NotificacionEmail(
            "ana@correo.com", "Bienvenida al sistema",
            "Registro exitoso", false));
        notificaciones.add(new NotificacionSMS(
            "3101234567", "Su codigo es 4589", "Claro"));
        notificaciones.add(new NotificacionPush(
            "usuario_01", "Tienes una nueva oferta",
            "MiTienda"));
        notificaciones.add(new NotificacionEmail(
            "pedro@correo.com", "Reporte mensual adjunto",
            "Informe Febrero 2026", true));

        double costoTotal = 0;
        for (Notificacion n : notificaciones) {
            n.enviar();
            costoTotal += n.calcularCosto();
            System.out.println("Fecha: " + n.getFechaFormateada());
        }
    }
}

```

```
        System.out.println("---");
    }

    System.out.printf("%nCosto total: $%,.0f COP%n", costoTotal);
}
}
```

Ejercicio Práctico 4 - Polimorfismo

Cree un sistema de medios de pago para una tienda en línea: Clase abstracta MedioPago con métodos abstractos procesarPago(double monto) y calcularComision(double monto). Clases hijas: TarjetaCredito (comisión 3.5%), PSE (comisión fija \$2,500 COP), y Efecty (comisión 1.5% + \$1,000 COP). Cree un método estático procesarOrden(ArrayList<MedioPago> pagos, double montoTotal) que divida el pago entre los medios disponibles. Muestre el detalle de cada transacción y el total de comisiones.

6. Ejercicio Integrador Final

Desafío: Sistema de Gestión de Parqueadero

Desarrolle un sistema completo que integre los cuatro pilares de la POO. Este ejercicio se entregará como proyecto de NetBeans funcional.

6.1 Requerimientos

Diseñe e implemente un sistema de gestión de parqueadero que cumpla con las siguientes especificaciones:

Abstracción

- Clase abstracta Vehiculo con atributos placa y propietario, y métodos abstractos calcularTarifa(int horas) y getTipo().
- Interface Registrable con métodos registrarEntrada() y registrarSalida().

Encapsulamiento

- Todos los atributos deben ser privados con getters/setters validados.
- La tarifa no puede ser negativa, la placa debe tener formato válido (ej: ABC-123).

Herencia

- Clases hijas: Automovil (tarifa: \$3,000/hora), Motocicleta (tarifa: \$1,500/hora), Bicicleta (tarifa: \$500/hora).
- Cada clase hija sobrescribe calcularTarifa() con su lógica propia.

Polimorfismo

- La clase Parqueadero maneja un `ArrayList<Vehiculo>` y opera sobre los vehiculos sin conocer su tipo específico.
- Metodos: `ingresarVehiculo()`, `retirarVehiculo()`, `generarReporte()` y `calcularIngresosTotales()`.

6.2 Criterios de Evaluación

Criterio	Ponderacion
Correcta aplicacion de Abstraccion (clases abstractas e interfaces)	20%
Encapsulamiento adecuado con validaciones	20%
Herencia bien estructurada con uso de super	20%
Polimorfismo funcional con ArrayList y ciclos	20%
Funcionalidad completa, codigo limpio y documentado	20%

7. Resumen de los Pilares de la POO

Pilar	Proposito	Palabras Clave	Ejemplo del Taller
Abstraccion	Modelar lo esencial, ocultar lo complejo	<code>abstract</code> , <code>interface</code>	<code>FiguraGeometrica</code> , <code>Notificacion</code>
Encapsulamiento	Proteger datos, controlar acceso	<code>private</code> , <code>getters/setters</code>	<code>CuentaBancaria</code> , <code>Estudiante</code>
Herencia	Reutilizar codigo, jerarquias	<code>extends</code> , <code>super</code>	<code>Empleado -> EmpleadoTC</code>
Polimorfismo	Un metodo, multiples comportamientos	<code>Override</code> , <code>sobrecarga</code>	<code>ArrayList<Notificacion></code>

SENA

Programa ADSO - Análisis y Desarrollo de Software

Éxitos en su aprendizaje!